
HEBCPF Solver Suite

Release 2026.07.15

Holomorphic-Embedding-Based Continuation Power-Flow All-Solutions Solver

v4 MEX and pure-MATLAB release

Compiled solver: HEBCPF_MEX_v4_20260715

Portable solver: HEBCPF_matlab_v4_20260715

Default choice: MEX v4 on supported Windows x64; pure MATLAB v4 otherwise

Checkpoint file: temp_result.mat

This overview is a release-level map. Each solver folder contains its own `HEBCPF_User_Guide.pdf` with installation, parameters, and resume details.

July 15, 2026

Contents

1	Release Scope	2
1.1	Relationship to earlier releases	2
2	Core v4 Features	2
3	Choosing A Solver	3
4	Benchmark Comparison	3
5	Choosing A Mode	4
6	Quick Start	4
7	Resuming A Ceased Search	5
8	Bundled Cases	5
9	Validation Summary	6
10	Release Contents	6
11	Citation	6

1 Release Scope

This release packages the v4 HEBCPF solver in two forms:

Folder	Solver	Use it when
HEBCPF_MEX_v4_20260715	Windows x64 MEX-accelerated v4	You want the fastest shipped build on a supported Windows MATLAB installation.
HEBCPF_matlab_v4_-20260715	Pure MATLAB v4	You need portability, easier inspection, or a no-compiler/no-MEX workflow.

Both solvers use the same HEB C formulation, MATPOWER-format cases, v4 numerical parameters, and user-facing drivers. They should not be placed on the MATLAB path at the same time because they contain functions with overlapping names.

1.1 Relationship to earlier releases

Release 2026.07.15 is a benchmarked maintenance release of the v4 line, not a new mathematical method. Relative to the 2026.07.14 v4 release, it retains the formulation, numerical defaults, compiled hot paths, and queue scheduler; the substantive driver change is serializable checkpoint/resume support for long `parfor` and queue `parfeval` runs, plus the retained MEX barrier driver. Earlier v2/v3 packages and the 2026.07.14 v4 package remain useful for reproducing historical results and are available through the corresponding GitHub tags. Do not mix solver folders from different releases on one MATLAB path.

2 Core v4 Features

- **Consensus Padé-pole selection.** The holomorphic-to-continuation hand-off uses a pole estimate agreed across several buses, reducing spurious single-bus pole choices.
- **Deterministic keyed deduplication.** Candidate solutions are matched with a fixed random signature and an L1 tolerance, avoiding the old linear scan over the full solution matrix.
- **slope_max = 4e5.** The v4 folders ship with the requested turning point slope threshold in `solver_params.m` and in the parallel runbooks.
- **Global parfeval work queue.** The recommended driver keeps one in-flight trace per equation and submits new work as soon as a worker finishes. This preserves VBook pruning while avoiding row-barrier idle time.
- **Checkpoint/resume support.** Long parallel searches periodically save `temp_result.mat`; resuming rebuilds futures, pools, and deduplication state.
- **Cached/vectorized hot path.** V4 reduces repeated sparse analysis, holomorphic assembly work, and Padé allocation overhead.

3 Choosing A Solver

Question	MEX v4	Pure MATLAB v4
Windows x64 MATLAB?	Recommended. Shipped <code>.mexw64</code> binaries are included.	Works, but generally slower.
Linux or macOS?	Rebuild MEX binaries first, or use pure MATLAB.	Recommended out of the box.
Need to inspect or modify hot-path code?	Possible, but rebuild if changing MEX source.	Recommended for experimentation.
Need maximum speed?	Recommended.	Use when portability matters more than speed.
Need no compiler?	Works on Windows x64 with shipped binaries.	Always works without a compiler.

4 Benchmark Comparison

The released benchmark compares the two shipped v4 packages, not earlier solver releases. It ran the queue `parfeval` driver at nominal load on all 20 bundled cases through 57 buses using MATLAB R2022a on PCWIN64 with 23 local workers. Parallel-pool startup (44.46 s) was performed once before the timed loop and is excluded; the reported per-case wall times include pre-processing and tracing. The benchmark driver is `run_benchmark_up_to_57bus_20260715.m`; the committed machine-readable results are in `benchmark_results_20260715/20260715_194046/benchmark_summary_all.csv`.

Solver	Cases	Total wall (s)	Trace wall (s)	Solutions
MEX v4	20	718.656	716.062	3256
Pure MATLAB v4	20	1272.966	1271.222	3256

Across the complete suite, the MEX package took $1.77\times$ less wall-clock time (43.5% less elapsed time) and was faster on 14 of 20 cases. This is a package-level comparison: MEX v4 uses the compiled Padé/holomorphic kernels and its shipped KLU path, whereas pure MATLAB v4 uses only `.m` code and its dense corrector. Very small cases are overhead-sensitive, so pure MATLAB was faster on six of them; do not treat the aggregate ratio as a platform-independent or serial-mode speedup.

Both solvers returned the same solution count for every case. A one-to-one, order-independent comparison of the saved solution sets gave a maximum nearest-solution distance of 3.154×10^{-7} , below the shared keyed-deduplication tolerance of $4e-7$.

Case	Buses	Solutions	MEX (s)	MATLAB (s)	Ratio
case3	3	6	1.128	0.621	0.55×
case3TS	3	6	0.313	0.332	1.06×
case4BB0	4	14	1.023	0.767	0.75×
case4BBc	4	12	2.025	0.409	0.20×
case4gs	4	6	0.352	0.356	1.01×
case5loop	5	10	0.595	0.834	1.40×
case5Salam	5	10	0.656	0.703	1.07×
case5Salam_mod3	5	4	0.424	0.307	0.72×

Case	Buses	Solutions	MEX (s)	MATLAB (s)	Ratio
case6ww	6	6	0.623	0.571	0.92×
case7Salam	7	4	0.479	0.456	0.95×
case9	9	8	0.904	0.922	1.02×
case9Q	9	8	0.546	0.700	1.28×
case14mod	14	30	3.110	4.495	1.45×
case14mod2	14	68	6.134	8.640	1.41×
case30	30	472	61.776	93.156	1.51×
case33bw	33	16	4.855	9.713	2.00×
case39	39	176	34.870	58.694	1.68×
case_ieee30	30	472	62.862	91.943	1.46×
case57mod	57	606	168.996	305.818	1.81×
case57	57	1322	366.985	693.529	1.89×

The ratio is MATLAB total wall time divided by MEX total wall time; values above one favour MEX. The full benchmark report explains the retained local logs and snapshots used for validation, which are intentionally not part of the GitHub release payload.

5 Choosing A Mode

Each solver has three main collection modes:

serial

Uses `runVBook_hybrid_2023.m`. It is the easiest mode to debug and needs no Parallel Computing Toolbox.

parfor

Uses `runVBook_hybrid_parallel.m`. It traces one row of VBook at a time and waits for all equations in that row to finish.

parfeval

Uses `runVBook_hybrid_parfeval.m`. It is asynchronous and keeps workers busy without waiting for the slowest equation in each row. This is the recommended mode for `case30`, `case39`, `case57mod`, `case57`, and larger systems.

The MEX folder also keeps `runVBook_hybrid_parfeval_barrier.m`. That file is a retained comparison driver, not the recommended default.

6 Quick Start

```
cd('<path>/HEBCPF_MEX_v4_20260715')          % or HEBCPF_matlab_v4_20260715
addpath(pwd)

which runpf
which makeYbus

[result, solutions] = run_merged_case('case14mod', 'parfeval');
result
```

Batch entry points:

Driver	Mode
run_batch.m	serial
run_batch_par.m	parfor
run_batch_parfeval.m	queue parfeval

7 Resuming A Ceased Search

The parallel v4 runbooks write periodic checkpoints to `temp_result.mat`. To resume:

1. Return to the same solver folder.
2. Use the same case, load factor, and driver mode.
3. Load `temp_result.mat`.
4. Run the matching runbook.

```
cd('<path>/HEBCPF_MEX_v4_20260715')      % or the pure MATLAB folder
addpath(pwd)
load temp_result.mat
wait = 0;
run('runVBook_hybrid_parfeval.m')      % queue parfeval checkpoint
% run('runVBook_hybrid_parallel.m')    % parfor checkpoint
```

For the MEX retained barrier driver:

```
load temp_result.mat
wait = 0;
run('runVBook_hybrid_parfeval_barrier.m')
```

For serial runs, save the workspace manually before stopping:

```
save('my_checkpoint.mat', '-v7.3')
load my_checkpoint.mat
wait = 0;
run('runVBook_hybrid_2023.m')
```

Internally, the resume path checks `VBook` and `Zsave`. If `Zsave` contains extra preallocated columns, it is trimmed to the book-kept solution count. The queue `parfeval` driver rebuilds pending work from `VBook == 0`; runtime futures and parallel-pool objects are not restored from the checkpoint.

8 Bundled Cases

case3 (6)	case4gs (6)	case4BB0 (14)	case4BBc (12)
case5loop (10)	case5Salam (10)	case6ww (6)	case7Salam (4)
case9 (8)	case9Q (8)	case14mod (30)	case33bw (16)
case39 (176)	case30/case_ieee30 (472)	case57mod (606)	case57 (1322)

Counts above are nominal-load reference counts for the completed bundled benchmarks, collapsing antipodal duplicates in the solver's canonical output convention.

9 Validation Summary

During preparation of this release, the v4 resume behavior was checked by comparing uninterrupted runs with forced checkpoint/resume runs:

Folder	Driver	Cases	Result
MEX v4	parallel, barrier	parfeval, case14mod, case39	Matching solution counts; set errors below $4e-7$.
Pure MATLAB v4	parallel, parfeval	case14mod	Matching solution counts; set errors below $4e-7$.

The user guides were checked against the actual runbooks for checkpoint name, resume path, driver names, and `slope_max = 4e5`.

10 Release Contents

Path	Purpose
README.md	GitHub front page and quick start.
CHANGELOG.md	Release history.
RELEASE_NOTES_20260715.md	GitHub release notes for this package.
RELEASE_CHECKLIST_-20260715.md	Final GitHub payload and local-verification checklist.
run_benchmark_up_to_-57bus_20260715.m	Reproducible queue-parfeval benchmark driver.
benchmark_results_-20260715/20260715_194046/	Committed benchmark report and combined CSV.
CITATION.cff	Software citation metadata.
LICENSE, NOTICE	License and third-party notices.
HEBCPF_Suite_-Overview.tex/pdf	This overview.
HEBCPF_MEX_v4_20260715/	Compiled Windows x64 v4 solver.
HEBCPF_matlab_v4_-20260715/	Pure MATLAB v4 solver.
klurf/	KLU refactor MEX source, build helper, and notes.

11 Citation

If you use results produced by this solver, please cite:

D. Wu and B. Wang, “Holomorphic Embedding Based Continuation Method for Identifying Multiple Power Flow Solutions,” *IEEE Access*, vol. 7, pp. 86843–86853, 2019. doi: [10.1109/ACCESS.2019.2925384](https://doi.org/10.1109/ACCESS.2019.2925384).

For citing the software package, use the included `CITATION.cff`.

HEBCPF Solver Suite 2026.07.15. This document accompanies the solver release and may be redistributed with it.